APEX: A Provenance-Controlled, Multi-Library Measurement of the Source-Attribution Ceiling for Stripped-Binary Functions

Syem Shibly Ador
Macquarie University
Sydney, Australia
syemshibly.ador@mq.edu.au

Siam Shibly Antar
McGill University
Montreal, Canada
siam.antar@mail.mcgill.ca

***Abstract*—Recovering the open-source origin of each function in a stripped binary would help software composition analysis, vulnerability triage, and license auditing, and a growing line of work pursues it by retrieving candidate source and verifying it, increasingly with language models. We ask a prior question: how many functions in a stripped binary even *can* be attributed by such methods? We answer it with APEX, a provenance-controlled, multi-library measurement. From a corpus we compile ourselves (recorded compiler and flags), with authoritative function-to-file ground truth from the unstripped builds, we measure at the binary level the *attributability ceiling* (CEIL): the share of functions carrying any compilation-invariant discriminator (a rare constant, a string, or a distinctive external call) that public code search can key on, an upper bound on what such lexical-feature retrieval can attribute. Across thirty C libraries the ceiling ranges from 0.0% to 84.0% at `-O2` (up to 72.4% at `-O3`, median near 32.5%), and twenty sit below 40%, far below what an SBOM or a CVE check would need and independent of any verifier. Because searchability is necessary but not sufficient, we tighten CEIL to an *attributable-in-principle* fraction, the share whose features identify a single source file; it sits 11.4 points under the ceiling (median 21.2%), and enlarging the reference from fifteen to thirty libraries lowers it further (25.2% to 22.3% on the shared libraries), so against all public code the gap can only widen, not close, by an amount we do not measure. The ceiling is near-stable under static linking (−5.5 points) and rises about 12 points under clang, with the broad cross-library ordering preserved. A magic-constant baseline is near-optimal where signal is constant-dominated (F1 up to 59.8%) and weak otherwise, and a worked-example pipeline sits at the band's floor. The limit is on per-function attribution, not on detecting that a library is present, which needs only one distinctive function. A second extractor on a different disassembler and ELF parser reproduces it exactly, so the ceiling is a property of the binaries, not the tool. We release the corpus, ground truth, extractor, and scorers.**

***Index Terms*—binary analysis, reverse engineering, software composition analysis, software provenance, decompilation, supply-chain security, measurement**

I. Introduction

A large fraction of compiled software embeds open-source components. When the software ships stripped, the symbol table that would name each routine is gone. Per-function attribution, mapping a binary routine to the upstream file it was compiled from, would let an SBOM list vendored and statically linked dependencies, let a responder decide whether a CVE applies, and substantiate license reuse.

Most prior work compares a query function to a known reference and asks whether they match [1]–[6]; reuse detectors report which component is present rather than which file a routine maps to [7]–[10]; symbol-recovery methods predict a name rather than cite a file [11]–[14]. A newer direction retrieves candidate source from public code and verifies it, sometimes with a language model [15], [16]. All presume the target function carries enough surviving signal to be found in the first place, a premise rarely examined.

We examine it directly with APEX (Attributability Profiling of stripped EXecutables), a provenance-controlled measurement. Rather than proposing a tool, APEX measures the ceiling that bounds *lexical-feature* source retrieval: the fraction of functions carrying any compilation-invariant discriminator (a rare constant, a string, or a distinctive call name) that public code search can key on. A function carrying none cannot be retrieved by its lexical features, however good the verifier; a method keyed on structure instead, a learned binary-to-source embedding or call-graph context, reads a signal the gate does not measure and falls outside CEIL. We then tighten this upper bound to the fraction *attributable in principle* and test how the band moves across build configurations. We make the following contributions.

- **A provenance-controlled, multi-library corpus.** Thirty C libraries we compile ourselves with recorded compiler and flags (one across four optimization levels), so every binary has a known build and authoritative function-to-file ground truth from its unstripped image.
- **A measured attributability ceiling (CEIL).** A reproducible objdump-based extractor gives, per binary, the fraction of functions clearing an identifiability gate: 0.0% to 84.0% at `-O2` (72.4% at `-O3`, median ≈32.5%), below 40% for twenty of thirty.
- **A measured band, not just a ceiling.** Tightening CEIL by intra-corpus feature uniqueness, the *attributable-in-principle* fraction sits a median 11.4 points below it (only 63% of searchable functions are file-unique); it is an upper bound that tightens as the reference grows (25.2%

to 22.3% on the shared libraries as the corpus doubles from fifteen to thirty), and is library-specific, from zero (xxHash) to about 39 points (klib).

- **Robustness across build axes.** The ceiling is near-stable under static linking (-5.5 points) and rises about 12 points under clang's heavier inlining, with the broad cross-library ordering preserved (Spearman $\rho = 0.83$), so attributability is build- as well as library-dependent.
- **A diagnostic baseline.** A magic-constant lookup (near-100% precision) recovers most searchable signal where it is constant-dominated (xxHash, 81% of the ceiling) but little otherwise (zlib, 11%); it marks where a discriminating method has room.
- **A worked example at the floor.** A tiered retrieval-and-verification pipeline (SRT) on a zlib binary attributes fewer than one function in ten, below both bounds and the baseline.
- **The ceiling is not a tool artifact.** A second extractor sharing only the gate, on Capstone and pyelftools instead of objdump and nm, reproduces CEIL exactly on all 2566 functions across the thirty libraries.

II. Related Work

Function-level similarity. Genius [1], Gemini [2], SAFE [3], jTrans [4], Trex [5], Asm2Vec [6], and structural diffing [17] compute similarity to a reference function; they presuppose a candidate to compare against. **Reuse and license auditing.** BAT [7], OSSPolice [8], B2SFinder [9], BinaryAI [15], and CENTRIS [10] detect components at file or package granularity. **Symbol recovery.** DEBIN [11], Punstrip [12], SymLM [13], and DIRTY [14] predict names or types. **LLMs for binaries.** Recent work applies language models to decompilation [16]; our worked example uses a local code model [18], [19] only to adjudicate retrieval. None measures a *method-independent* ceiling on what is attributable at all: BinaryAI's recall bound reflects one embedding model's capability, and Dramko et al. [20] measure how obfuscation degrades specific name-recovery models, not whether discriminative signal survives compilation. CEIL is that ceiling.

III. Measurement Method

A. Provenance-Controlled Corpus

We compile each library ourselves and record version, compiler, exact flags, and the SHA-256 of the stripped output. We select widely used, permissively licensed C libraries chosen to span library types rather than to sample a population: the corpus is thirty libraries (2566 functions), spanning compression (zlib 1.3.1 [21], LZ4, miniz, heatshrink), hashing and checksums (xxHash, sha2), cryptography (monocypher), JSON and config parsing (cJSON, yyjson, parson, jsmn, tomlc99, inih), markdown (md4c), serialization (mpack), encoding (base64, qrcodegen), text, string, and path utilities (utf8proc, sds, cwalk), regular expressions (tiny-regex-c), expression evaluation (tinyexpr), networking and protocol parsing (mongoose, http_parser, picohttpparser), data structures (klib), logging (log.c), image decoding (stb), tar archives (microtar), and a scripting core (Lua's core VM), all with `gcc 13.3.0` at `-O2`, plus zlib at `-O0`/`-O2`/`-O3`/`-Os`. Compiling ourselves, rather than analyzing a found binary, is what lets us state ground truth without guessing. To probe build sensitivity, we also recompile with `clang 18.1.3` and model static linking by recomputing CEIL with the external-call signal (absent without a PLT) disabled.

Algorithm 1 Identifiability gate (a function is searchable if not NONE)

Require: rare-constant count c, string/data-ref count s, external-call count e

if $c \geq 1 \wedge s \geq 1$ **then return** HIGH
else if $c \geq 2 \vee s \geq 2$ **then return** HIGH
else if $c \geq 1 \vee s \geq 1$ **then return** MEDIUM
else if $e \geq 2$ **then return** MEDIUM
else if $e \geq 1$ **then return** LOW
else return NONE
end if

B. Ground Truth and the File-Role Criterion

From each unstripped build we derive an authoritative address-to-function-to-file map: per-object symbol tables give the source file, the image gives the address. An attribution is *file-role correct* if a returned candidate shares the canonical filename of the function's true definition. Recall is over the full eligible inventory, so a function never searched counts as a miss.

C. Binary-Level Features and the Attributability Ceiling

The quantity we measure is how many functions are searchable at all. We extract, per function, three classes of compilation-invariant signal directly from the compiled object with objdump and nm: rare numeric constants (immediates after removing common and architectural values and applying a magnitude threshold, plus a catalog of known magic constants), references into read-only data (a proxy for string and table use), and external calls (resolved through the PLT). A function clears the identifiability gate (Algorithm 1) if it carries at least one such discriminator, and the *attributability ceiling* (CEIL, Ceiling Estimation for Inferred Lineage) is the fraction of a binary's functions that clear it. The HIGH/MEDIUM/LOW tiers are inherited from the SRT pipeline, which uses them to order search; only the NONE versus not-NONE boundary enters CEIL. CEIL upper-bounds the recall of any *lexical-feature* retrieval method, one searching on constants, strings, or call names as public code search does, under a perfect verifier and perfect retrieval, because functions below it carry none of these. It does not bound a method keyed on structure, a learned binary-to-source embedding or call-graph context, which reads a signal the gate does not measure and can match a feature-less function. Working from the binary rather than from source is deliberate: macros and precomputed tables hide constants in source that are present in the compiled image (the CRC-32 table, for instance), so a source-level estimate undercounts.

D. From Searchable to Attributable-in-Principle

A discriminator enables attribution only if it is globally distinctive: a moderately rare constant or a generic string that recurs across many files cannot pin a function to its origin. We tighten CEIL with an intra-corpus uniqueness test: for each searchable function we collect its identifiable features (rare-constant values, named const-table symbols, content fingerprints of referenced read-only data, and distinctive external callees) and index each against the source files carrying it corpus-wide. A feature is *distinctive* if every function bearing it belongs to a single source file (by basename, so vendored copies count as one), and a function is *attributable-in-principle* if it carries at least one. A finite reference over-counts distinctiveness, so this fraction is an *upper* bound that tightens as the reference grows: we report it against the full thirty-library corpus, and recomputing the original fifteen libraries against a fifteen- versus thirty-library reference shows the tightening directly, with miniz colliding with zlib on shared CRC and deflate constants. The estimate errs on both sides, the objdump extractor missing references it cannot resolve while the finite reference inflates it. The band geometry is strict, CEIL $\geq$ attributable-in-principle $\geq$ realizable: retrieval returns collisions for any non-distinctive feature, so a realizable method sits below this bound, not between it and CEIL.

E. Magic-Constant Baseline

As a floor we use a constant lookup: a function is attributed if it embeds a globally-unique cataloged constant (CRC-32 polynomial, Adler modulus, published xxHash primes, LZ4 frame magics). Such constants are unambiguous, so the baseline's precision is essentially 100%; its recall is whatever fraction of a library's functions carry one.

F. Worked Example: a Retrieval-and-Verification Pipeline

To show what an actual pipeline achieves relative to the ceiling, we use SRT (Fig. 1): it gates functions by the same identifiability test, builds compilation-invariant code-search queries, verifies each candidate with a locally hosted code model, Qwen2.5-Coder-7B served by Ollama [18], [19], and propagates confirmed matches along the call graph. Read as an example, not a claim: verification is local, but the search stage sends distinctive constants and strings to a third-party API, so the binary is not kept private.

IV. Results

A. The Attributability Ceiling Across Libraries

Fig. 2 shows the thirty libraries at `-O2`, and Table I reports all thirty-three builds. The ceiling ranges from 0.0% (base64, whose alphabet table is mutable and so excluded by the read-only-data proxy) to 84.0% (qrcodegen) at `-O2`, reaching 72.4% for zlib at `-O3`, with a median near 32.5% and twenty of thirty below 40%. The pattern is interpretable: compression, hashing, checksum, and 2D-barcode code (zlib, xxHash, qrcodegen) and table-driven text processing (utf8proc) carry distinctive constants or large named tables and sit near the top, while parsers, serializers, and small string, path, and archive utilities (parson, mpack, microtar, base64) carry mostly generic control flow and ubiquitous calls and sit near the bottom. Attributability is therefore not a property of a method but largely of the library. Because the corpus spans library types rather than sampling a population, the median and the "twenty below 40%" count describe the represented types, not functions in the wild.

B. From Searchable to Attributable

Fig. 2 overlays the attributable-in-principle fraction on CEIL (the Attrib. column of Table I). Across the thirty libraries it sits 11.4 points below CEIL in the median (32.5% versus 21.2%); per-library gaps run from zero to about 39 points, median 8.0, and only 63% of searchable functions carry a globally distinctive feature, the rest clearing the gate on constants, jump tables, or generic data that recur across files. This is an upper bound that over-counts distinctiveness: on the original fifteen libraries the median attributable falls from 25.2% to 22.3% once the reference doubles to thirty, and zlib falls from 28.5% to 22.3% as miniz, an independent deflate reimplementation, contributes the same CRC and deflate constants from within the corpus. The direction generalizes, more reference code yields fewer file-unique features, so against all public code the gap can only widen, not close, by an amount we do not measure. The gap is diagnostic: zero for xxHash and http_parser, whose prime constants and protocol tables are unique, and among the widest for zlib and klib, the textbook searchable-but-not-attributable case. The bound is on *lexical-feature* retrieval; a method keyed on the call graph, as our worked example is, can attribute a featureless function from confirmed neighbors and so sits below this line, still under true attributability.

C. The Baseline Tracks Constant-Saturation

The constant baseline tracks constant-saturation, not the ceiling: its recall is near-optimal where signal is unique-constant dominated and small otherwise. On xxHash, saturated with prime magics, the matcher reaches about 81% of the ceiling (recall 42.6% against 52.5%, F1 59.8%), so a heavier pipeline has little to add. On zlib it inverts: most searchable functions are reachable through strings, tables, and calls, so the baseline captures roughly one-ninth of the ceiling (6.2% against 58.5%) and leaves about 52 points of headroom; LZ4 falls between at roughly 22% (11.1% against 49.4%). Because the applications weight precision over recall, a precision-favoring score widens its advantage wherever it fires, and the headroom it leaves is where a discriminating method can operate.

D. A Pipeline Operating Below the Ceiling

We ran the SRT pipeline end to end once, on a separately obtained zlib shared object: scored by the file-role criterion it attributes 9 of 107 eligible functions (precision 28.1%, recall 8.4%, F1 12.9%). This is a worked example, not a measured comparison, since the binary was labeled zlib 1.2.11 with its build unrecorded and is not directly commensurable with the controlled 1.3.1 corpus. Even read loosely it sits far below the zlib ceiling ($\approx$59%) and loses to the constant baseline on a precision-weighted basis, leaving most attributable headroom unused.

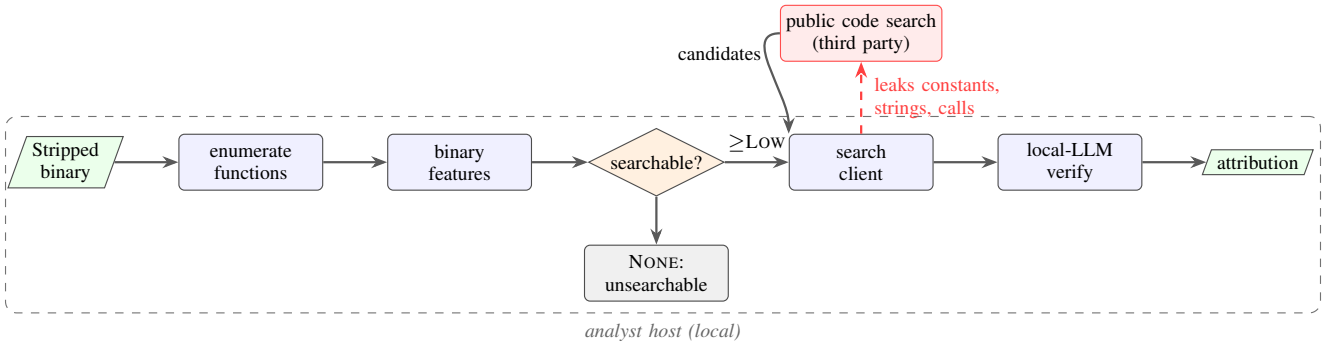

Fig. 1. The worked-example pipeline. The same identifiability gate used for the ceiling measurement decides which functions are searched. All stages are local except code search, whose queries leak the binary's distinctive features.

TABLE I

MEASURED BINARY-LEVEL ATTRIBUTABILITY CEILING (CEIL) AND ATTRIBUTABLE-IN-PRINCIPLE FRACTION (ATTRIB., JUDGED AGAINST THE FULL THIRTY-LIBRARY CORPUS) FOR THIRTY C LIBRARIES AT GCC `-O2`, WITH THE MAGIC-CONSTANT BASELINE (RECALL / F1; PRECISION ≈100% BY CONSTRUCTION, "–" MEANS NO CATALOGED CONSTANT APPLIES). LIBRARIES ARE ORDERED BY CEILING; THE LOWER BLOCK IS THE ZLIB OPTIMIZATION SWEEP. ALL VALUES REPRODUCIBLE FROM THE RELEASED KIT.

Library	N	Ceil.	Attr.	Base. R/F1	Library	N	Ceil.	Attr.	Base. R/F1
base64	7	0.0%	0.0%	–	mongoose	265	33.2%	24.5%	–
mpack	366	6.8%	1.6%	–	picohttpparser	9	33.3%	11.1%	–
parson	120	9.2%	5.8%	–	tinyregex	8	37.5%	0.0%	–
cJSON	90	10.0%	7.8%	–	stb	113	38.1%	23.9%	–
tinyexpr	27	11.1%	3.7%	–	miniz	141	39.7%	28.4%	1.4% / 2.8%
cwalk	33	12.1%	9.1%	–	monocypher	82	41.5%	32.9%	–
tomlc99	57	12.3%	5.3%	–	LZ4	162	49.4%	31.5%	11.1% / 20.0%
heatshrink	16	12.5%	0.0%	–	jsmn	2	50.0%	50.0%	–
inih	7	14.3%	0.0%	–	md4c	60	50.0%	36.7%	–
microtar	19	15.8%	0.0%	–	xxHash	61	52.5%	52.5%	42.6% / **59.8%**
sha2	5	20.0%	20.0%	–	http_parser	17	52.9%	52.9%	–
sds	50	22.0%	20.0%	–	zlib	130	58.5%	22.3%	6.2% / 11.6%
logc	8	25.0%	25.0%	–	utf8proc	33	60.6%	48.5%	–
yyjson	68	25.0%	19.1%	–	klib	177	61.6%	22.6%	–
Lua	408	31.9%	22.8%	–	qrcodegen	25	84.0%	80.0%	–

zlib optimization sweep: `-O0`: N=152, CEIL 50.0%, base. 2.0%/3.9% `-O3`: N=123, CEIL 72.4%, base. 6.5%/12.2% `-Os`: N=137, CEIL 48.9%, base. 2.2%/4.3%

E. Optimization Sensitivity

On zlib the ceiling moves with optimization, from 50.0% at `-O0` to 72.4% at `-O3`, with `-Os` lowest at 48.9% (Table I): inlining shrinks the inventory while more of the survivors carry a distinctive constant or table, so the searchable count rises. The swing is real but narrower than the cross-library spread, confirming attributability is build-dependent even within one library.

F. Robustness across Build Axes

Two build axes test whether the ceiling is a property of the code or of our gcc, dynamically linked configuration. Static linking removes the PLT and the external-call signal; disabling that signal lowers the median by only 5.5 points (32.5% to 27.0%), since after filtering ubiquitous calls the searchable signal is carried by constants and tables (a first-order model isolating the call-signal effect, not a real static binary, which also pulls libc in). The compiler matters more: under clang the median rises about 12 points (32.5% to 44.5%) as heavier inlining shrinks the inventory (zlib from 130 to 81 functions) and packs signal into fewer functions, and the attributable edge rises with it (21.2% to 26.6%), so the whole band shifts up rather than only its top. The broad cross-library ordering is largely preserved (Spearman $\rho = 0.83$), so attributability is build- as well as library-dependent.

G. Sensitivity of the Ceiling

CEIL depends on three extractor choices, all swept in Fig. 3. The rare-constant threshold has little leverage: across a 256-fold range it moves the median by 10.7 points (37.9% to 27.2%), since most functions clear the gate through a string, table, or distinctive call rather than a borderline constant. The call-filtering choice dominates: not filtering ubiquitous calls lifts the median from 32.5% to 64.0%, a 31.4-point swing, because a function whose only external call is to `malloc` or `memcpy` then clears the gate; it is the largest lever by about three to one, so the canonical measurement filters these calls. Third, with jump tables already discounted (an indirect-jump target is not counted as data), restricting the rest to named const tables lowers the canonical median by 7.5 points, the share reachable only through anonymous read-only tables the extractor does not name.

H. Component Detection versus Per-Function Attribution

Per-function attribution and library-presence detection are different questions, and the ceiling answers the first. Detecting presence needs only one attributable function; attributing every

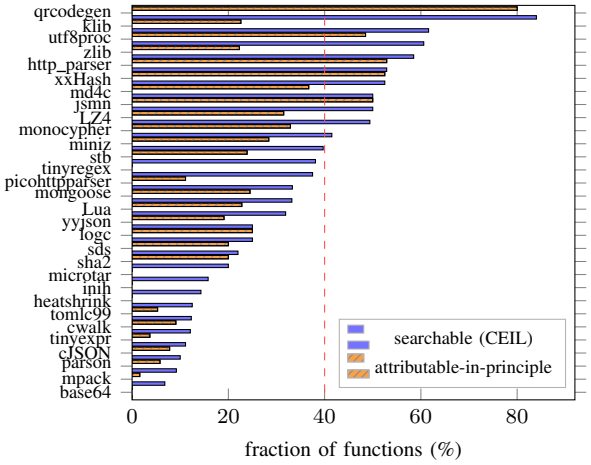

Fig. 2. The attributability band per library (gcc `-O2`) across the thirty-library corpus: the searchable ceiling (CEIL) and the attributable-in-principle fraction beneath it. The gap is the share searchable only through non-distinctive features. It is near zero for hashing and HTTP code, whose constants and tables are unique, and among the widest for zlib and klib, which are searchable largely through generic deflate and hash-table constants that recur elsewhere in the corpus. The attributable line uses the thirty-library reference and is an upper bound; an enlarged reference lowers it further. The dashed line marks 40%.

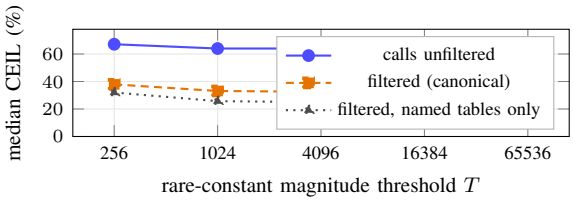

Fig. 3. Sensitivity of the median ceiling across the thirty libraries. The rare-constant threshold (x-axis, log scale) moves the median by 10.7 points, and restricting data references to named const tables by 7.5; filtering ubiquitous libc and runtime calls moves it by 31.4. The canonical setting filters calls and counts both named and anonymous (jump-table-discounted) tables.

function needs them all. A library with N functions, A of them file-attributable, that contributes k functions to a binary is detectable in principle with probability $1 - \binom{N-A}{k}/\binom{N}{k}$, an upper bound for the same reason the ceiling bounds attribution and treating the draw as uniform (real dead-code elimination is reachability-driven). At $k = 1$ this equals the per-function attributable fraction; once the whole library links it is one whenever $A \geq 1$. Fig. 4 plots it. The median per-function attributable fraction is 21.2%, but ten linked functions include a distinctive one with median probability 92.8%, and 25 of the 30 fully linked libraries carry at least one. Five small libraries (base64, heatshrink, inih, microtar, and tinyregex) carry no file-attributable function at all, so they are undetectable in principle by this signal even when fully linked. The function behind CVE-2022-37434, a heap over-read in zlib's `inflate`, stays file-attributable even with miniz, an independent deflate reimplementation, in the corpus, so the affected component is locatable; pinning the specific vulnerable version is a strictly harder bar the file-level ceiling does not address. Per-function attribution is fundamentally limited; detecting a present component in principle, the target of tools such as OSSPolice

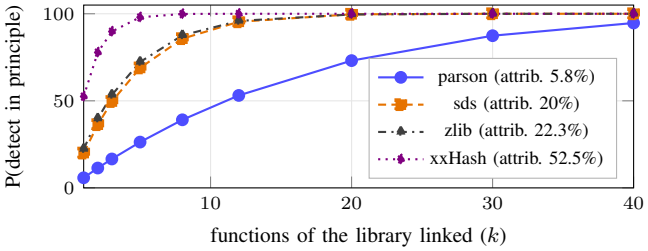

Fig. 4. Probability that a present library is detectable in principle as a function of how many of its functions the binary links, for four libraries spanning the ceiling range. The legend gives each library's attributable fraction, where its curve begins at $k = 1$; detection saturates well before a library is fully linked even when that fraction is low, because presence needs only one distinctive function.

and CENTRIS, is not, and the two diverge because presence needs a single distinctive function while attribution needs the whole set.

I. *Independent-Extractor Validation*

To test whether the ceiling reflects the binaries or how we read objdump's output, we re-derive CEIL with a second pipeline sharing only the gate: it disassembles with Capstone [22] and reads symbols and relocations with pyelftools [23], replacing objdump and nm, while the rare-constant test, magic catalog, libc filter, and jump-table discount are imported unchanged. The two agree on every one of the 2566 functions across all thirty libraries (mean absolute difference 0.0 points), so CEIL is a property of the compiled code under a fixed gate, not of the objdump text interface. The released kit runs both.

V. Discussion

Attributability is bounded and library-dependent. For twenty of thirty libraries the ceiling sits below 40%, and below 15% for the smallest parsers and utilities. No verifier can attribute a function that carries no lexical signal, so these ceilings cap every lexical-feature method; one keyed on structure or call-graph context, as our worked example is, reads a different signal and is the exception. Per-function attribution is thus most effective on constant-rich code and weak on the parsing and glue code that dominates many binaries.

Searchable is not attributable. Over a third of searchable functions carry only non-distinctive signal, and for zlib the searchable fraction is more than twice the attributable one; enlarging the reference only widens the gap. Reported as a single number, searchability overstates what attribution can achieve, and the gap names where a method is retrieval-limited rather than verification-limited.

The baseline is competitive where signal is constant-dominated. The ceiling rests on constants: removing the rare-constant signal drops the median by 12.3 points, against 1.2 for data references and 0 for distinctive calls. A constant lookup is therefore near-optimal where constants dominate, and a heavier pipeline, which our worked example did not beat, has room only where they do not.

VI. Limitations and Future Work

Our extractor approximates a decompiler's feature set from objdump and nm, a conservative proxy, and the gate admits any single discriminator, so the ceilings carry two-sided error quantified by the sensitivity sweep (Fig. 3). Four directions remain. *Deeper validation*: the Capstone/pyelftools extractor already reproduces CEIL exactly, so a decompiler-based extractor (Ghidra [24] or IDA) would test the feature set itself, and an oracle test of whether any NONE-tier function is attributable from structure would confirm the gate is a true floor. *The realizable rung*: a controlled retrieval-and-verification experiment over a fixed candidate set of true source plus distractors, with an oracle variant, would locate where a strong method lands within the band and separate retrieval- from verification-limited loss. *Breadth*: the corpus is thirty x86-64 C libraries spanning many types but still weighted toward small ones, so other instruction sets (aarch64) and formats (PE) need an extractor port, and a larger stratified corpus with real stripped artifacts (sqlite, openssl, busybox, firmware) would turn this type-spanning distribution into a population-level estimate. *Version attributability*: because the distinctiveness test collapses by basename it answers which file, whereas CVE triage needs which version, so the share of file-attributable functions carrying release-pinning features is the harder quantity the application requires. The released harness supports each; none changes the central result that most parsing and glue code is not attributable by content.

VII. Conclusion

With APEX we measured how attributable the functions of a stripped binary are, across thirty C libraries with controlled provenance and ground truth. The searchable ceiling is strongly library-dependent (0.0% to 84.0% at `-O2`, 72.4% at `-O3`, median ≈32.5%) and below 40% for most; intra-corpus uniqueness tightens it to an attributable-in-principle bound 11.4 points lower, and lower still as the reference grows from fifteen to thirty libraries. The band is stable under static linking and shifts upward under clang, and a worked-example pipeline sits at its floor. Across the types we sample, lexical-feature attribution is often fundamentally limited, and that limit, not verifier quality, should frame future work.

Reproducibility and Artifacts

We release the APEX kit: provenance-pinned gcc and clang build scripts, the ground-truth extractor, the ceiling, uniqueness, baseline, sensitivity, decomposition, and component-detection analyzers, the independent Capstone/pyelftools cross-check, and the file-role scorer. Every number in Figs. 2, 3, and 4 and Table I reproduces from the released tags, at https://github.com/syemador/apex.

References

[1] Q. Feng, R. Zhou, C. Xu, Y. Cheng, B. Testa, and H. Yin, "Scalable Graph-Based Bug Search for Firmware Images," in *Proc. ACM SIGSAC Conf. Computer and Communications Security (CCS)*, 2016.

[2] X. Xu, C. Liu, Q. Feng, H. Yin, L. Song, and D. Song, "Neural Network-Based Graph Embedding for Cross-Platform Binary Code Similarity Detection," in *Proc. ACM SIGSAC Conf. Computer and Communications Security (CCS)*, 2017.

[3] L. Massarelli, G. A. Di Luna, F. Petroni, R. Baldoni, and L. Querzoni, "SAFE: Self-Attentive Function Embeddings for Binary Similarity," in *Proc. Int. Conf. Detection of Intrusions and Malware, and Vulnerability Assessment (DIMVA)*, 2019.

[4] H. Wang, W. Qu, G. Katz, W. Zhu, Z. Gao, H. Qiu, J. Zhuge, and C. Zhang, "jTrans: Jump-Aware Transformer for Binary Code Similarity Detection," in *Proc. ACM SIGSOFT Int. Symp. Software Testing and Analysis (ISSTA)*, 2022.

[5] K. Pei, Z. Xuan, J. Yang, S. Jana, and B. Ray, "Trex: Learning Execution Semantics from Micro-Traces for Binary Similarity," *IEEE Trans. Dependable and Secure Computing*, 2023.

[6] S. H. H. Ding, B. C. M. Fung, and P. Charland, "Asm2Vec: Boosting Static Representation Robustness for Binary Clone Search Against Code Obfuscation and Compiler Optimization," in *Proc. IEEE Symp. Security and Privacy (S&P)*, 2019.

[7] A. Hemel, K. T. Kalleberg, R. Vermaas, and E. Dolstra, "Finding Software License Violations Through Binary Code Clone Detection," in *Proc. Working Conf. Mining Software Repositories (MSR)*, 2011.

[8] R. Duan, A. Bijlani, M. Xu, T. Kim, and W. Lee, "Identifying Open-Source License Violation and 1-Day Security Risk at Large Scale," in *Proc. ACM SIGSAC Conf. Computer and Communications Security (CCS)*, 2017.

[9] Z. Yuan, M. Feng, F. Li, G. Ban, Y. Xiao, S. Wang, Q. Tang, H. Su, C. Yu, J. Xu *et al.*, "B2SFinder: Detecting Open-Source Software Reuse in COTS Software," in *Proc. IEEE/ACM Int. Conf. Automated Software Engineering (ASE)*, 2019.

[10] S. Woo, S. Lee, H. Park, and H. Lee, "CENTRIS: A Precise and Scalable Approach for Identifying Modified Open-Source Software Reuse," in *Proc. IEEE/ACM Int. Conf. Software Engineering (ICSE)*, 2021.

[11] J. He, P. Ivanov, P. Tsankov, V. Raychev, and M. Vechev, "Debin: Predicting Debug Information in Stripped Binaries," in *Proc. ACM SIGSAC Conf. Computer and Communications Security (CCS)*, 2018.

[12] J. Patrick-Evans, L. Cavallaro, and J. Kinder, "Probabilistic Naming of Functions in Stripped Binaries," in *Proc. Annu. Computer Security Applications Conf. (ACSAC)*, 2020.

[13] X. Jin, K. Pei, J. Y. Won, and Z. Lin, "SymLM: Predicting Function Names in Stripped Binaries via Context-Sensitive Execution-Aware Code Embeddings," in *Proc. ACM SIGSAC Conf. Computer and Communications Security (CCS)*, 2022.

[14] Q. Chen, J. Lacomis, E. J. Schwartz, C. Le Goues, G. Neubig, and B. Vasilescu, "Augmenting Decompiler Output with Learned Variable Names and Types," in *Proc. USENIX Security Symp.*, 2022.

[15] L. Jiang, J. An, H. Huang, Q. Tang, S. Nie, S. Wu, and Y. Zhang, "BinaryAI: Binary software composition analysis via intelligent binary source code matching," in *Proc. IEEE/ACM Int. Conf. Software Engineering (ICSE)*, 2024.

[16] H. Tan, Q. Luo, J. Li, and Y. Zhang, "LLM4Decompile: Decompiling Binary Code with Large Language Models," in *Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP)*, 2024.

[17] T. Dullien and R. Rolles, "Graph-Based Comparison of Executable Objects," in *Proc. Symp. sur la Sécurité des Technologies de l'Information et des Communications (SSTIC)*, 2005.

[18] B. Hui, J. Yang, Z. Cui, J. Yang, D. Liu, L. Zhang, T. Liu, J. Zhang, B. Yu, K. Lu *et al.*, "Qwen2.5-Coder Technical Report," Alibaba Group, Tech. Rep., 2024, arXiv:2409.12186.

[19] Ollama Contributors, "Ollama: Get Up and Running with Large Language Models Locally," https://ollama.com, 2024, accessed: Jun. 17, 2026.

[20] L. Dramko, D. Bölöni-Turgut, C. L. Goues, and E. J. Schwartz, "Quantifying and mitigating the impact of obfuscations on machine-learning-based decompilation improvement," in *Proc. Int. Conf. Detection of Intrusions and Malware, and Vulnerability Assessment (DIMVA)*, 2025.

[21] J.-l. Gailly and M. Adler, "zlib: A Massively Spiffy Yet Delicately Unobtrusive Compression Library," https://zlib.net, 2024, version 1.3.1.

[22] N. A. Quynh, "Capstone: The Ultimate Disassembly Framework," https://www.capstone-engine.org, accessed: Jun. 17, 2026.

[23] E. Bendersky, "pyelftools: Parsing ELF and DWARF in Python," https://github.com/eliben/pyelftools, accessed: Jun. 17, 2026.

[24] National Security Agency, "Ghidra Software Reverse Engineering Framework," https://ghidra-sre.org, 2019, accessed: Jun. 17, 2026.